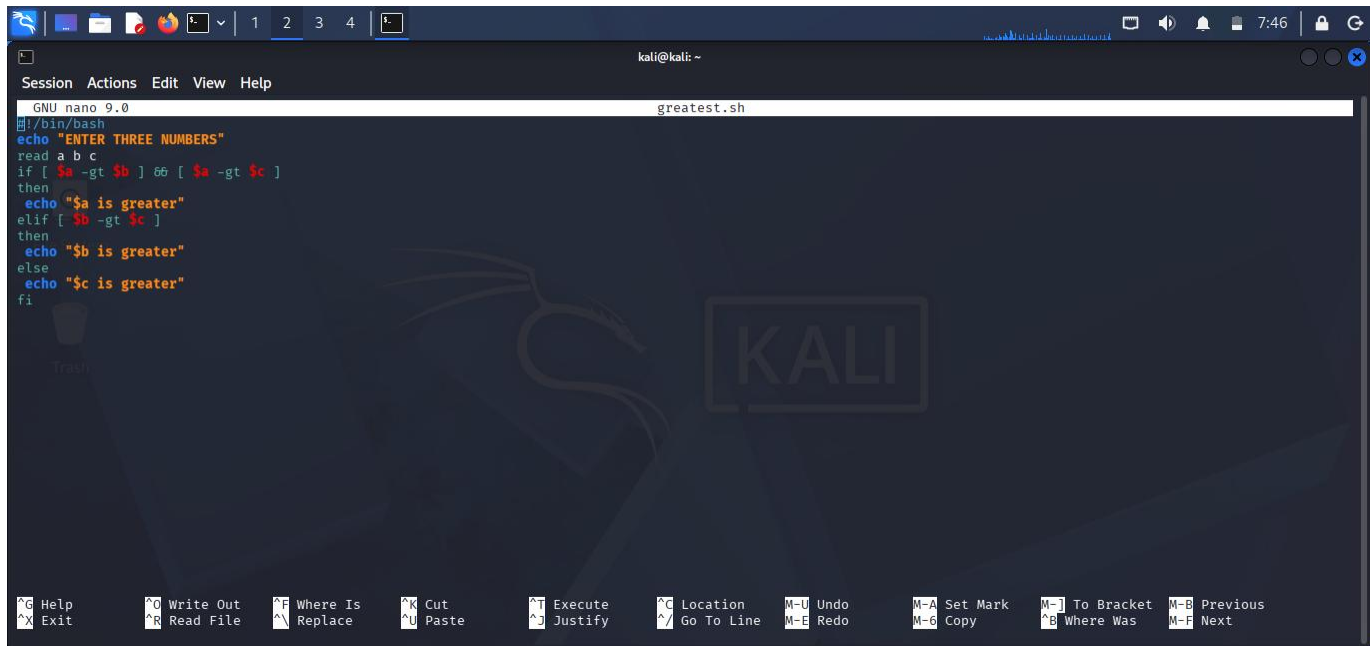


SHELL PROGRAMMING :GREATEST AMONG THREE NUMBERS

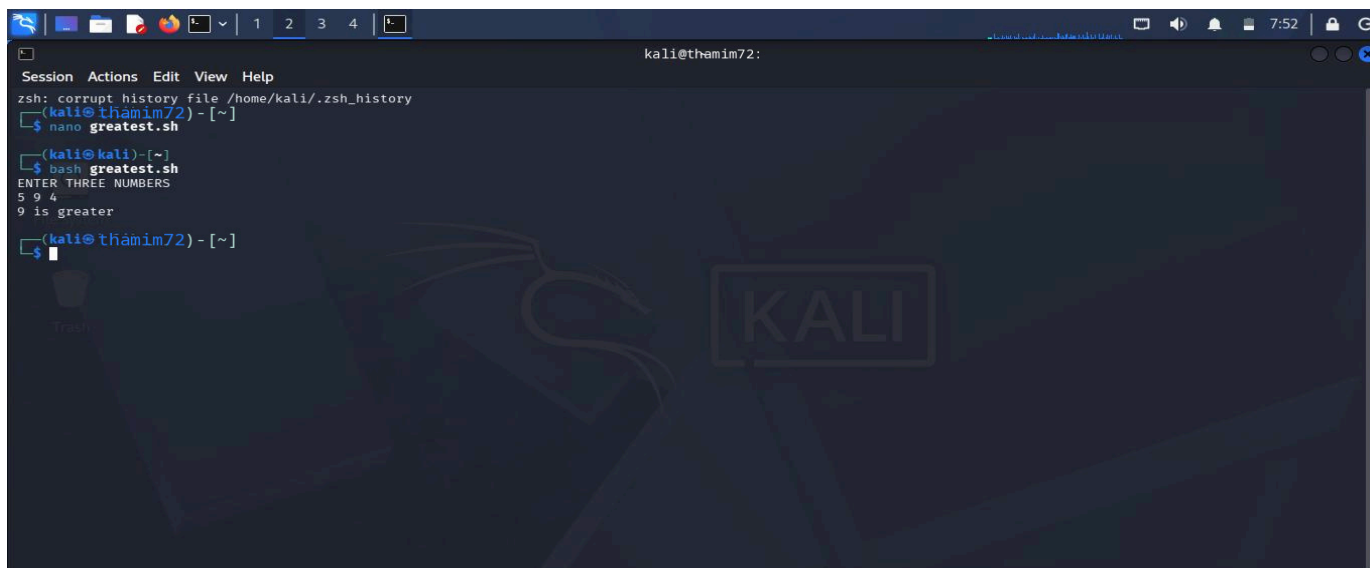


The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 'greatest.sh'. The script content is as follows:

```
#!/bin/bash
echo "ENTER THREE NUMBERS"
read a b c
if [ $a -gt $b ] && [ $a -gt $c ]
then
echo "$a is greater"
elif [ $b -gt $c ]
then
echo "$b is greater"
else
echo "$c is greater"
fi
```

The terminal window title is 'kali@kali: ~'. The nano editor's status bar at the bottom shows 'GNU nano 9.0 greatest.sh'. The bottom of the window displays various keyboard shortcuts for nano, such as 'Ctrl-G Help', 'Ctrl-X Exit', 'Ctrl-O Write Out', etc.

OUTPUT :

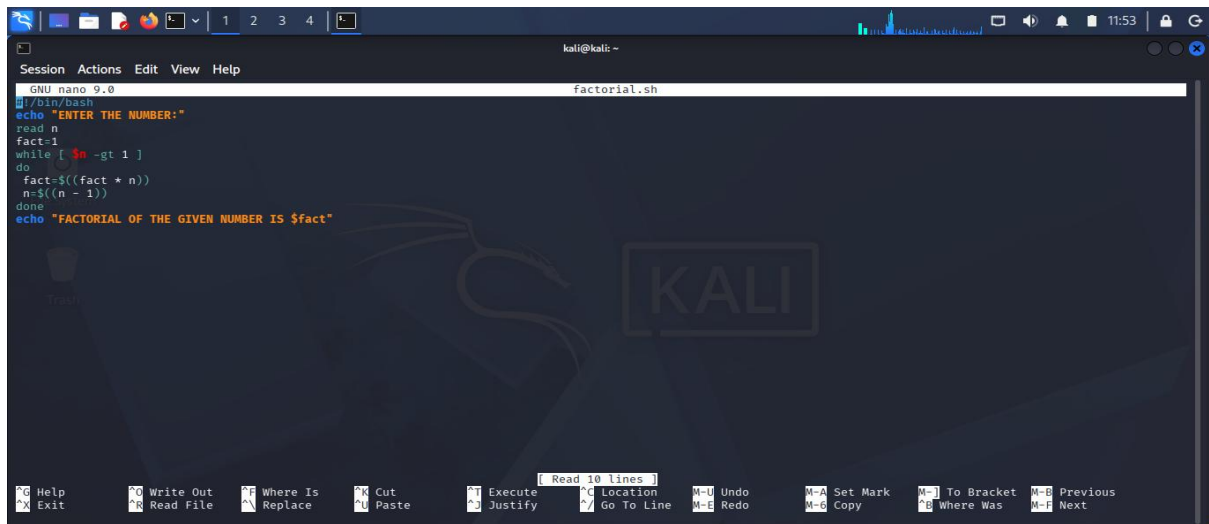


The screenshot shows a terminal window with the following output:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72)-[~]
$ nano greatest.sh
(kali@kali)-[~]
$ bash greatest.sh
ENTER THREE NUMBERS
5 9 4
9 is greater
(kali@thamim72)-[~]
$
```

The terminal window title is 'kali@thamim72:'. The prompt is '(kali@thamim72)-[~]'. The user has entered 'nano greatest.sh' and then 'bash greatest.sh'. The script has prompted for 'ENTER THREE NUMBERS', and the user has entered '5 9 4'. The script has outputted '9 is greater'.

SHELL PROGRAMMING :FACTORIAL OF A GIVEN NUMBER

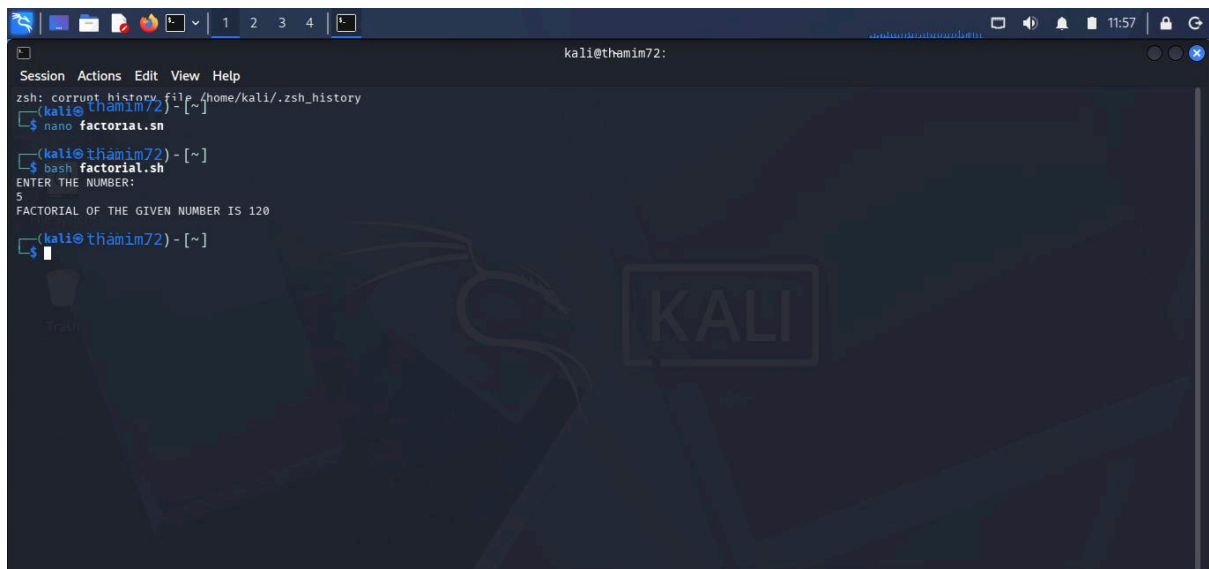


The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 'factorial.sh'. The script content is as follows:

```
#!/bin/bash
echo "ENTER THE NUMBER:"
read n
fact=1
while [ $n -gt 1 ]
do
    fact=$((fact * n))
    n=$((n - 1))
done
echo "FACTORIAL OF THE GIVEN NUMBER IS $fact"
```

The terminal window title is 'kali@kali: ~'. The nano editor's status bar at the bottom shows various keyboard shortcuts like 'Help', 'Exit', 'Write Out', 'Read File', etc.

OUTPUT :

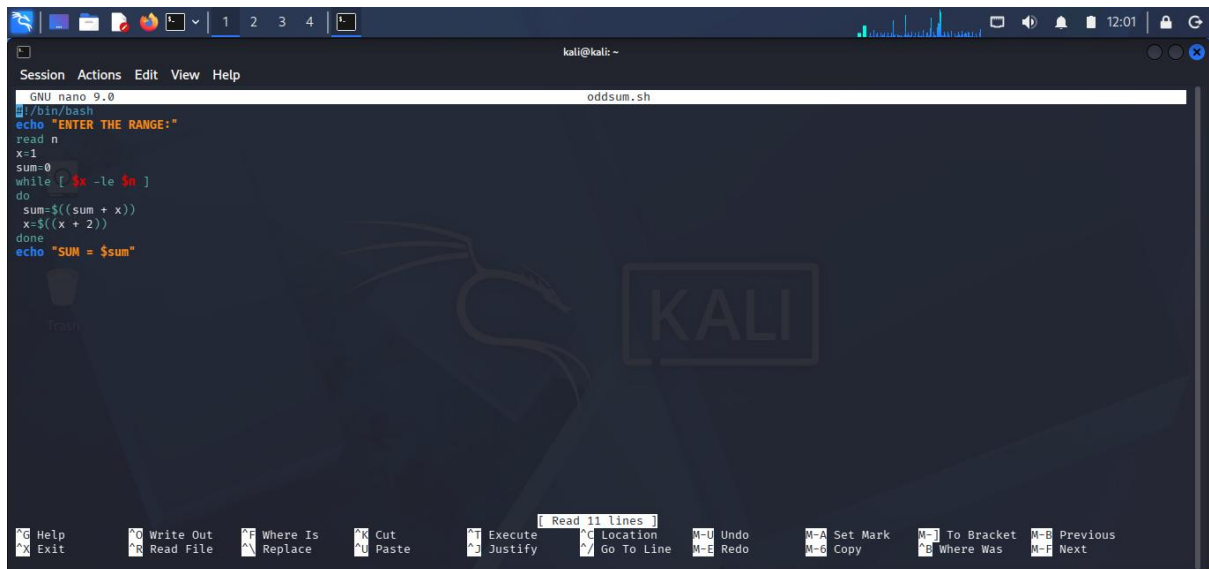


The screenshot shows a terminal window with the prompt 'kali@themim72:'. The user has entered the command 'nano factorial.sh' and then 'bash factorial.sh'. The output of the script is shown:

```
zsh: corrupt history file /home/kali/.zsh_history
kali@themim72) ~]
$ nano factorial.sh
kali@themim72) ~]
$ bash factorial.sh
ENTER THE NUMBER:
5
FACTORIAL OF THE GIVEN NUMBER IS 120
kali@themim72) ~]
$
```

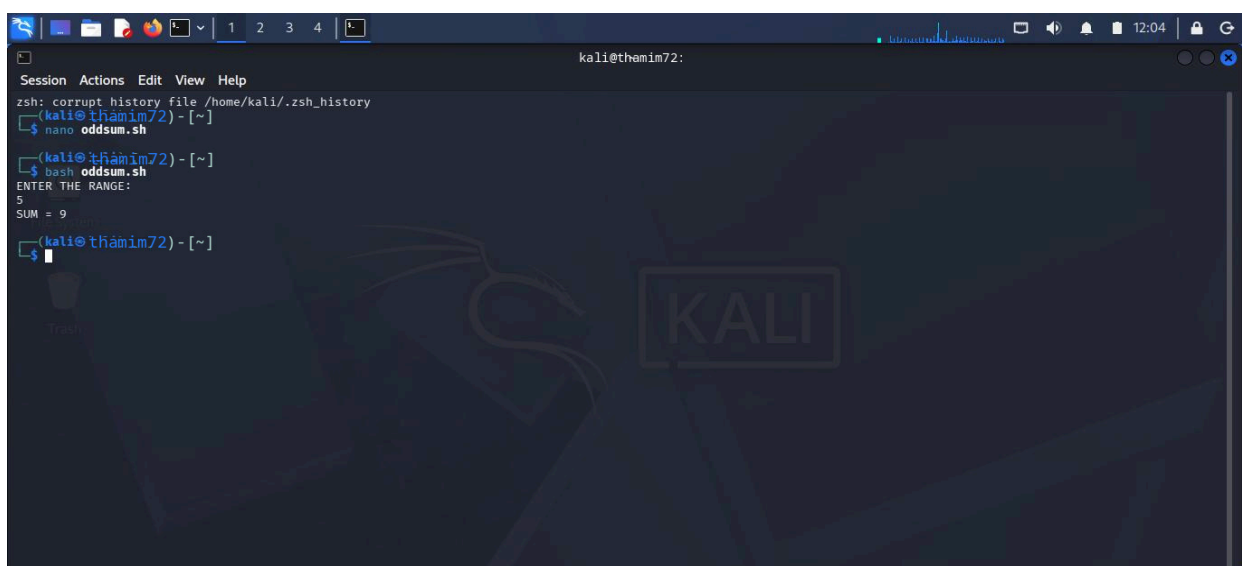
The terminal window title is 'kali@themim72:'. The background of the terminal has a Kali Linux logo watermark.

SHELL PROGRAMMING :SUM OF ODD NUMBERS



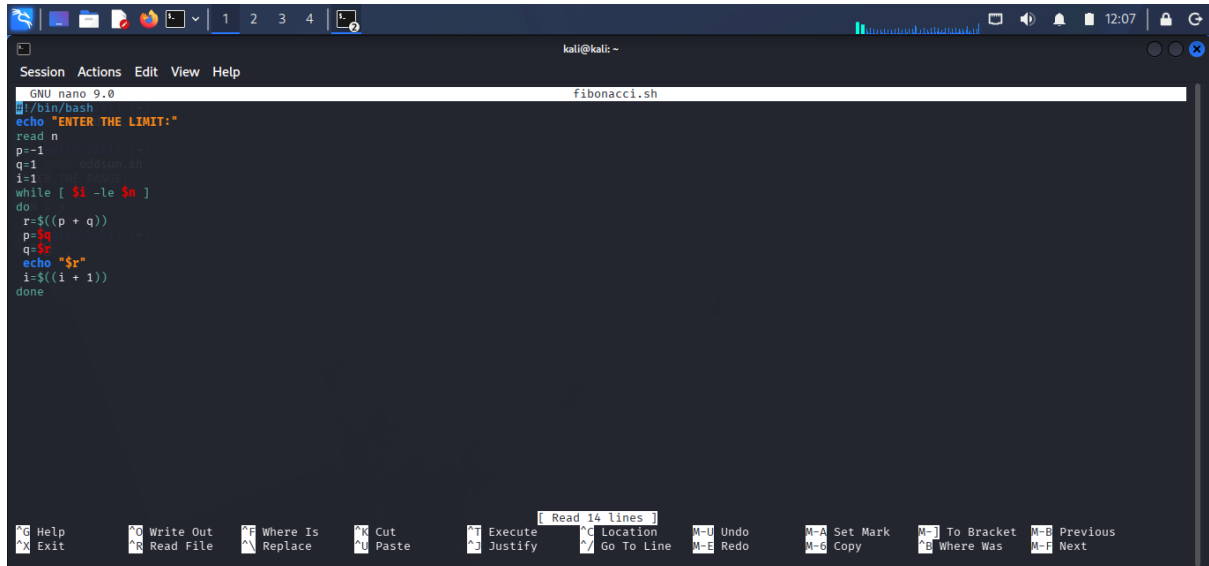
```
GNU nano 9.0 oddsum.sh
#!/bin/bash
echo "ENTER THE RANGE:"
read n
sum=0
x=1
while [ $x -le $n ]
do
    sum=$((sum + x))
    x=$((x + 2))
done
echo "SUM = $sum"
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72) - [~]
$ nano oddsum.sh
(kali@thamim72) - [~]
$ bash oddsum.sh
ENTER THE RANGE:
5
SUM = 9
(kali@thamim72) - [~]
$
```

SHELL PROGRAMMING :FIBONACCI NUMBER

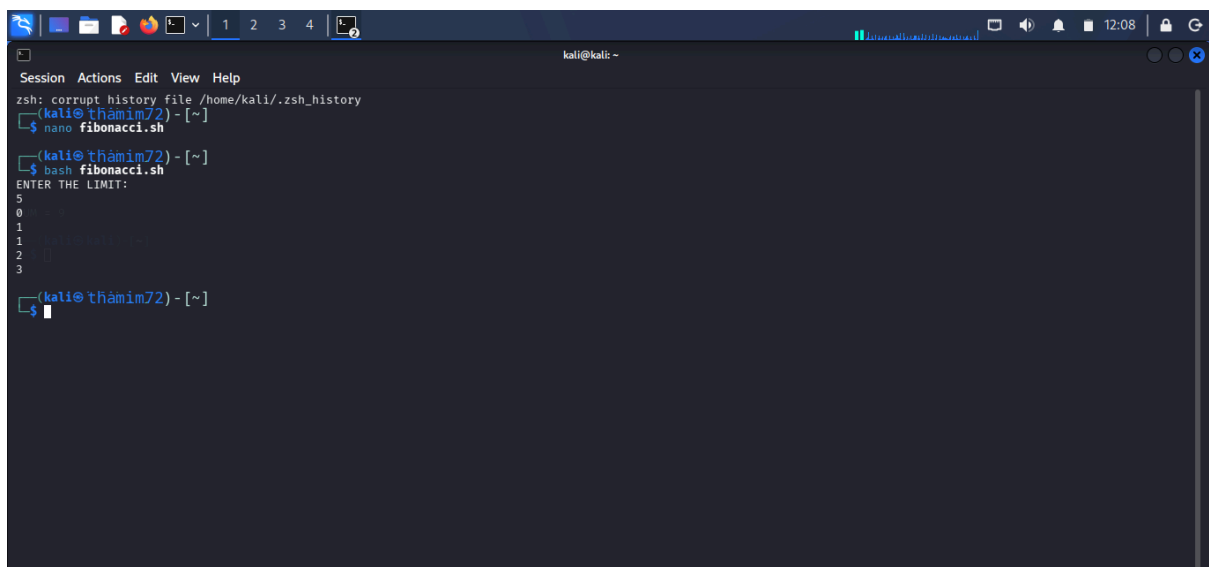


A screenshot of a terminal window on a Kali Linux system. The window title is "kali@kali: ~". The terminal shows the nano 9.0 editor editing a file named "fibonacci.sh". The script content is as follows:

```
#!/bin/bash
echo "ENTER THE LIMIT:"
read n
p=1
q=1
i=1
while [ $i -le $n ]
do
r=$((p + q))
p=$q
q=$r
echo "$r"
i=$((i + 1))
done
```

The nano editor's status bar at the bottom indicates "Read 14 lines". The bottom of the terminal window shows various nano editor shortcuts like Help, Exit, Write Out, Read File, Where Is, Replace, Cut, Paste, Execute, Justify, Location, Go To Line, Undo, Redo, Set Mark, Copy, To Bracket, Where Was, Previous, and Next.

OUTPUT :

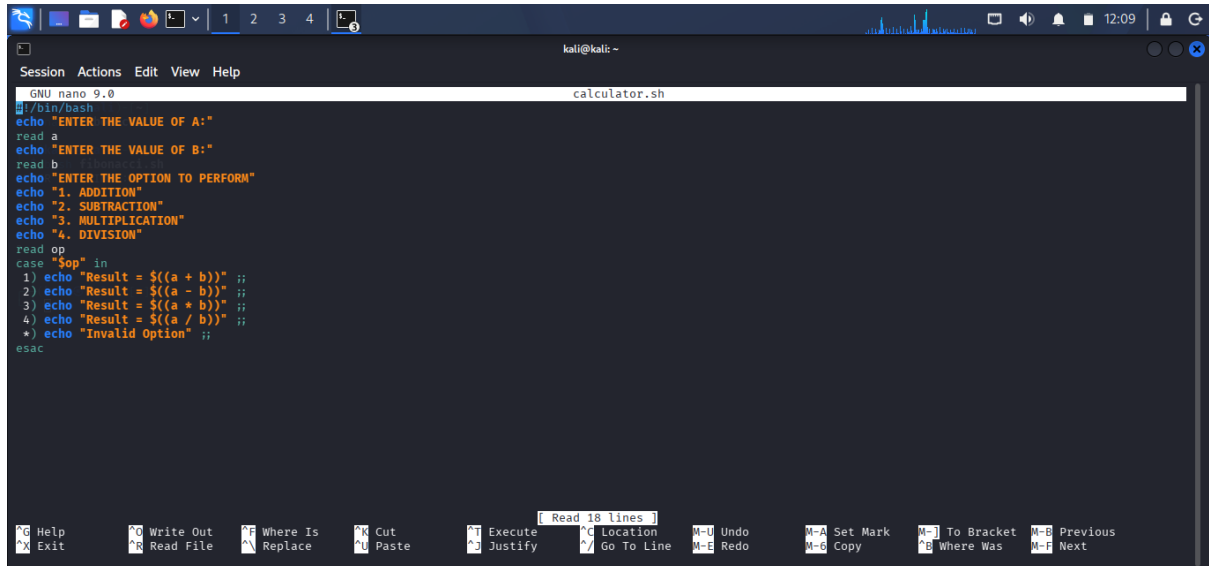


A screenshot of a terminal window on a Kali Linux system. The window title is "kali@kali: ~". The terminal shows the execution of the "fibonacci.sh" script. The output is as follows:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72) - [~]
$ nano fibonacci.sh
(kali@thamim72) - [~]
$ bash fibonacci.sh
ENTER THE LIMIT:
5
0
1
1
2
3
(kali@thamim72) - [~]
$
```

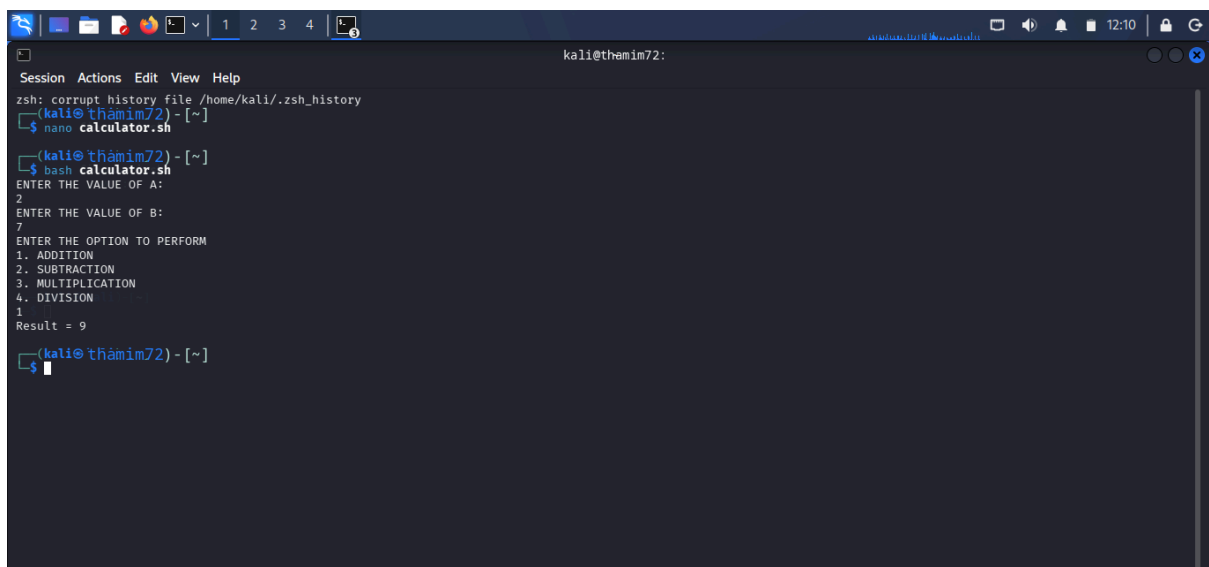
The terminal shows the user entering the limit "5" and the script outputting the Fibonacci sequence: 0, 1, 1, 2, 3.

SHELL PROGRAMMING :ARITHMETIC CALCULATOR



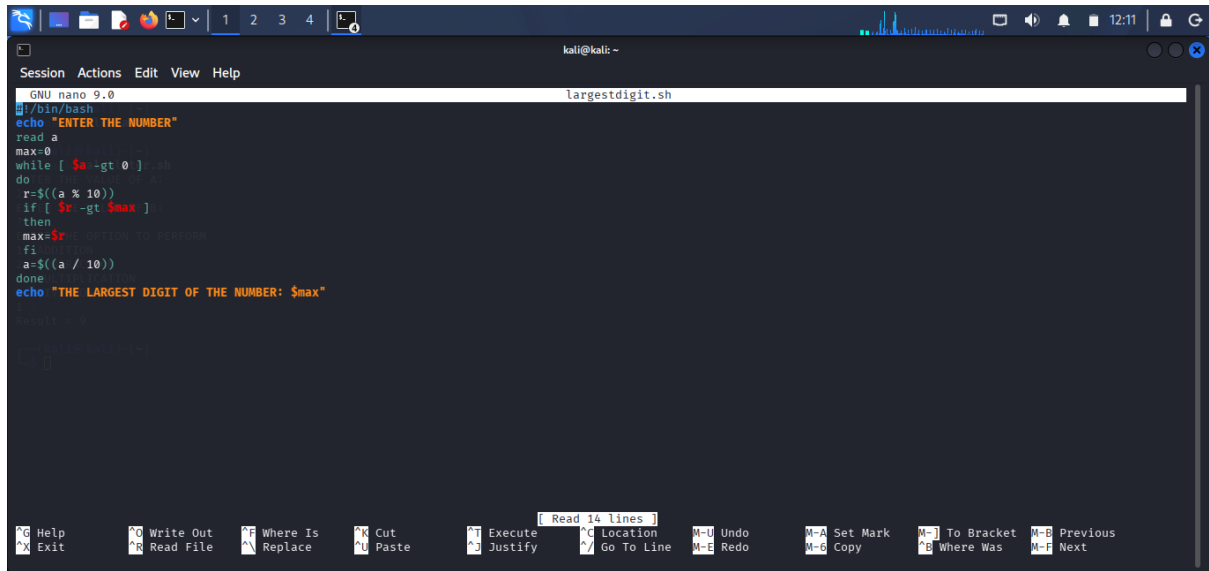
```
GNU nano 9.0 calculator.sh
#!/bin/bash
echo "ENTER THE VALUE OF A:"
read a
echo "ENTER THE VALUE OF B:"
read b
echo "ENTER THE OPTION TO PERFORM"
echo "1. ADDITION"
echo "2. SUBTRACTION"
echo "3. MULTIPLICATION"
echo "4. DIVISION"
read op
case "$op" in
1) echo "Result =  $$(a + b)$ " ;;
2) echo "Result =  $$(a - b)$ " ;;
3) echo "Result =  $$(a * b)$ " ;;
4) echo "Result =  $$(a / b)$ " ;;
*) echo "Invalid Option" ;;
esac
```

OUTPUT :



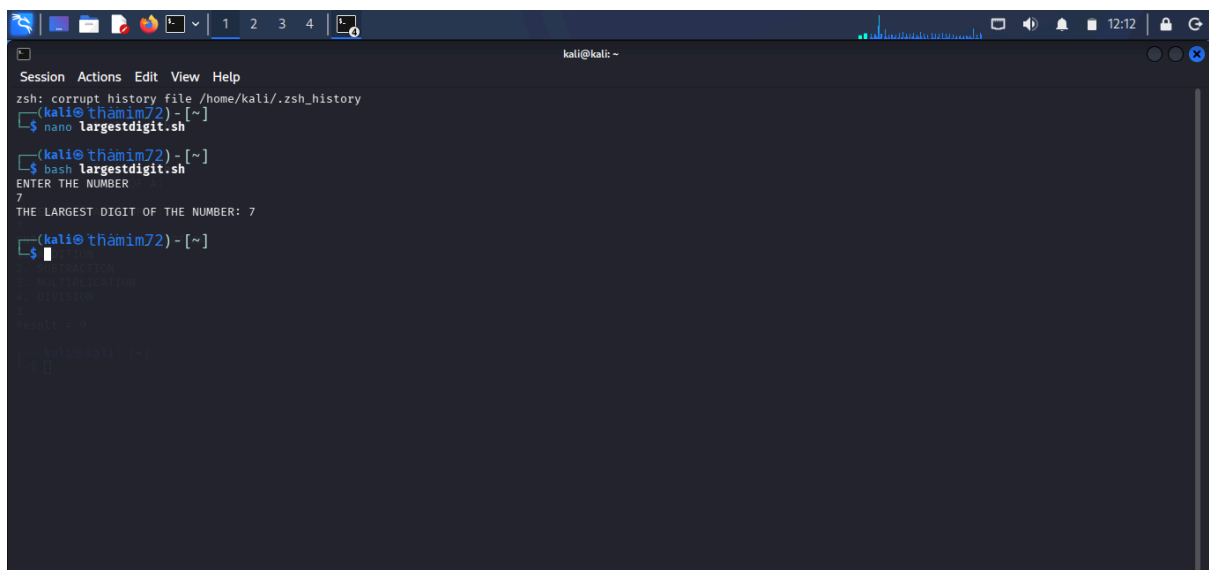
```
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72) - [~]
$ nano calculator.sh
(kali@thamim72) - [~]
$ bash calculator.sh
ENTER THE VALUE OF A:
2
ENTER THE VALUE OF B:
7
ENTER THE OPTION TO PERFORM
1. ADDITION
2. SUBTRACTION
3. MULTIPLICATION
4. DIVISION
1
Result = 9
(kali@thamim72) - [~]
$
```

SHELL PROGRAMMING :LARGEST DIGIT OF A NUMBER



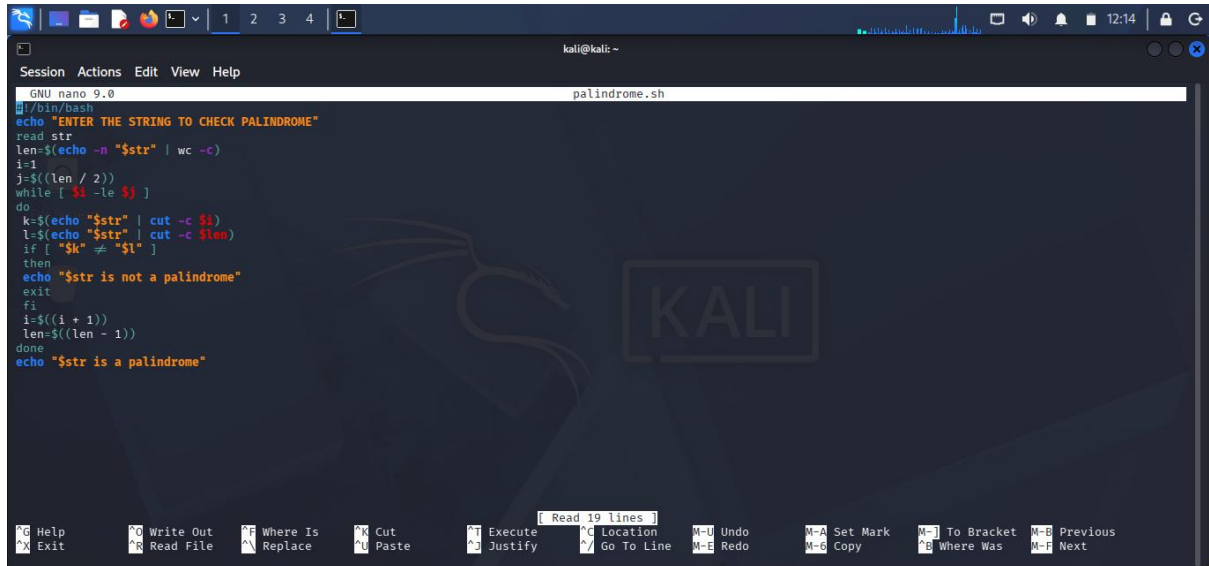
```
GNU nano 9.0 largestdigit.sh
#!/bin/bash
echo "ENTER THE NUMBER"
read a
max=0
while [ $a -gt 0 ]
do
r=$((a % 10))
if [ $r -gt $max ]
then
max=$r
fi
a=$((a / 10))
done
echo "THE LARGEST DIGIT OF THE NUMBER: $max"
```

OUTPUT :



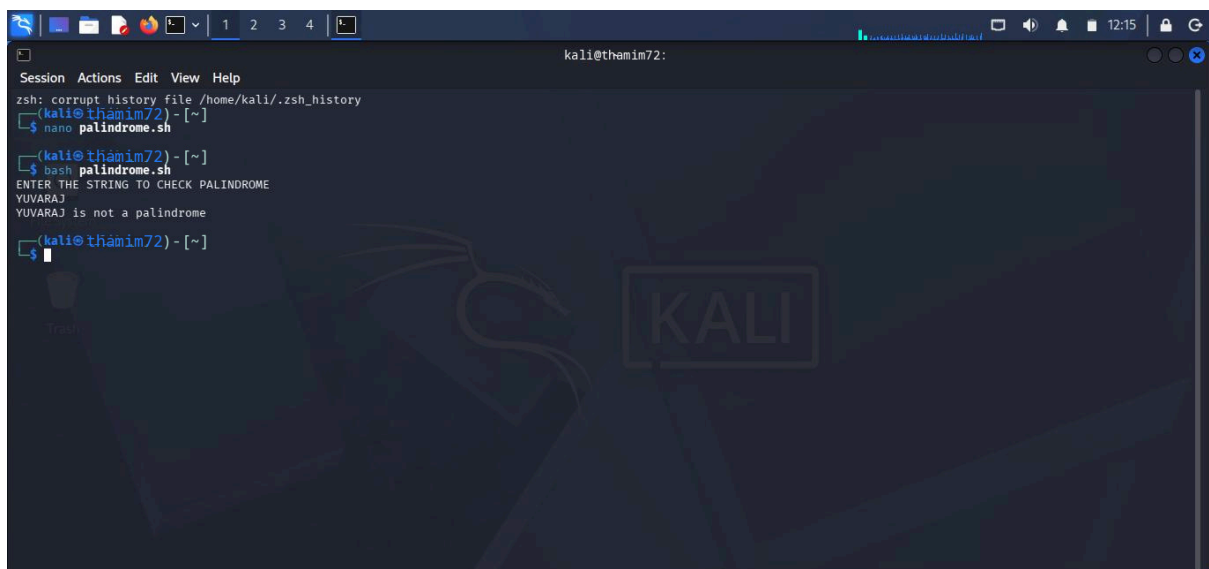
```
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72) - [~]
$ nano largestdigit.sh
(kali@thamim72) - [~]
$ bash largestdigit.sh
ENTER THE NUMBER
7
THE LARGEST DIGIT OF THE NUMBER: 7
(kali@thamim72) - [~]
$
```

SHELL PROGRAMMING :PALINDROME OR NOT



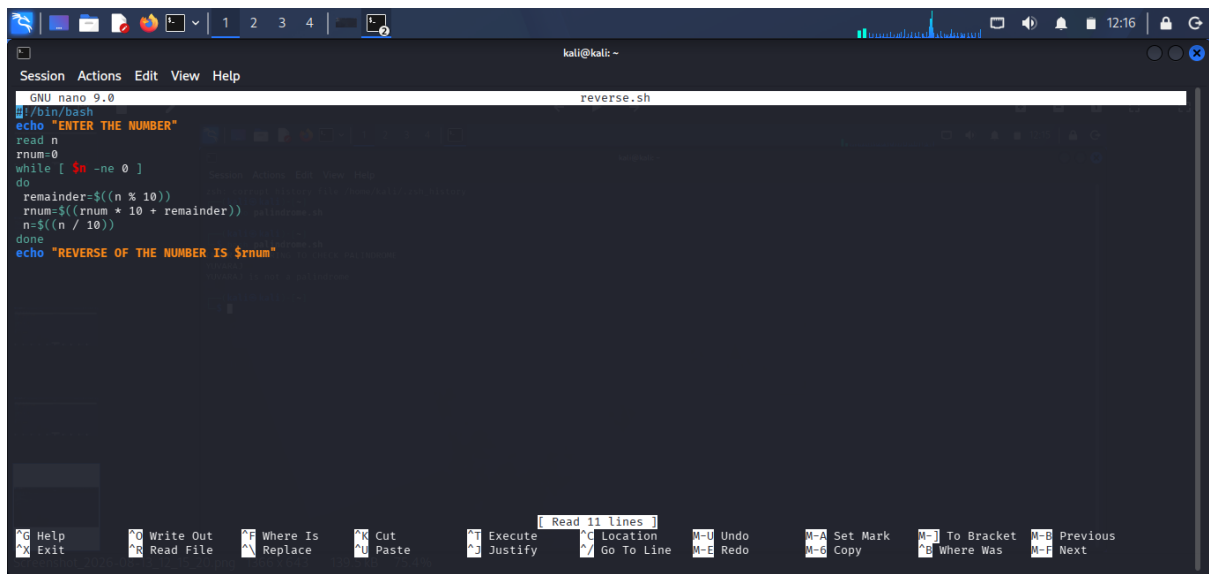
```
GNU nano 9.0
#!/bin/bash
echo "ENTER THE STRING TO CHECK PALINDROME"
read str
len=$(echo -n "$str" | wc -c)
i=1
j=$((len / 2))
while [ $i -le $j ]
do
k=$(echo "$str" | cut -c $i)
l=$(echo "$str" | cut -c $len)
if [ "$k" != "$l" ]
then
echo "$str is not a palindrome"
exit
fi
i=$((i + 1))
len=$((len - 1))
done
echo "$str is a palindrome"
```

OUTPUT :



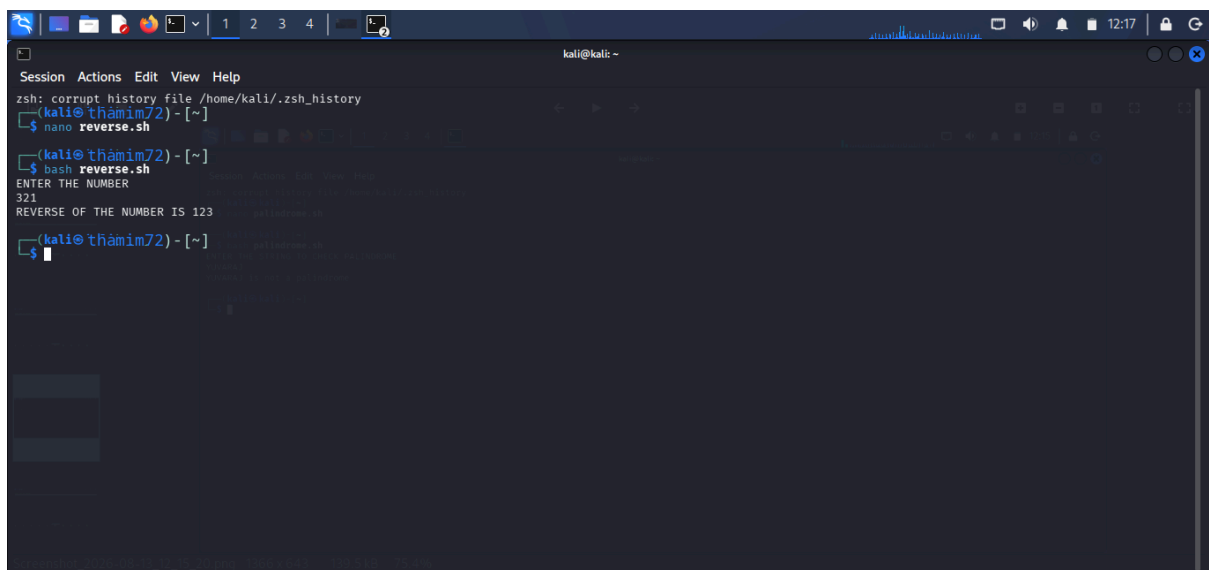
```
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72) - [~]
$ nano palindrome.sh
(kali@thamim72) - [~]
$ bash palindrome.sh
ENTER THE STRING TO CHECK PALINDROME
YUVARAJ
YUVARAJ is not a palindrome
(kali@thamim72) - [~]
$
```

SHELL PROGRAMMING :REVERSE OF A GIVEN NUMBER



```
GNU nano 9.0 reverse.sh
#!/bin/bash
echo "ENTER THE NUMBER"
read n
rnum=0
while [ $n -ne 0 ]
do
    remainder=$((n % 10))
    rnum=$((rnum * 10 + remainder))
    n=$((n / 10))
done
echo "REVERSE OF THE NUMBER IS $rnum"
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72) - [~]
$ nano reverse.sh
(kali@thamim72) - [~]
$ bash reverse.sh
ENTER THE NUMBER
321
REVERSE OF THE NUMBER IS 123
(kali@thamim72) - [~]
$
```